

SymPy Bug Report

1 Bug 55: singularities Misses $\text{polylog}(1, x)$ Singularity

1.1 Minimal Reproducer

```
from sympy import S, log, polylog, symbols
from sympy.calculus.singularities import singularities

x = symbols("x")
expr = polylog(1, x)
sing = singularities(expr, x, S.Complexes)

print("expr =", expr)
print("singularities =", sing)
print("contains 1 =", sing.contains(1))
print("value at 1 =", expr.subs(x, 1))
print("rewritten value at 1 =", (-log(1 - x)).subs(x, 1))
```

1.2 Observed Behavior

```
expr = polylog(1, x)
singularities = EmptySet
contains 1 = False
value at 1 = zoo
rewritten value at 1 = zoo
```

SymPy reports no singularities for $\text{polylog}(1, x)$ over the complex domain. The same run also shows that both the original expression and the logarithmic rewrite evaluate to `zoo` at $x = 1$, so the returned $\emptyset$ misses a concrete singular point.

1.3 Expected Behavior

The singularity set should include $x = 1$. In set notation,

$\text{singularities}(\text{polylog}(1, x), x, \mathbb{C})$ should contain $\{1\}$.

1.4 Mathematical Explanation

For order 1, the polylogarithm has the standard identity

$$\text{Li}_1(z) = \text{polylog}(1, z) = -\log(1 - z).$$

Therefore

$$\text{polylog}(1, x) = -\log(1 - x),$$

which has a logarithmic singularity when the logarithm argument vanishes:

$$1 - x = 0 \iff x = 1.$$

This is not a harmless alternative representation or branch-choice artifact. Returning $\emptyset$ asserts that there is no singular point in the complex domain, while substitution at $x = 1$ gives `zoo`, and the logarithmic identity exposes the missing singularity directly.

1.5 Source-Level Diagnosis

The diagnosis identifies the relevant public entry point as `singularities(expr, x, S.Complexes)` in `sympy/calculus/singularities.py`. The traced call path stays inside that function: it rewrites a small set of trigonometric and hyperbolic functions, scans the rewritten expression for negative-power `Pow` atoms, and then scans the original expression for a hard-coded set of logarithmic and inverse-hyperbolic atoms.

The key source-level behavior is that the scan only handles forms such as `Pow`, `log`, `asech`, `acsch`, `atanh`, and `acoth`. It has no handler for `polylog` and does not apply `expand_func`. The existing implementation of `polylog._eval_expand_func` in `sympy/functions/special/zeta_functions.py` already knows the order-one rewrite $\text{polylog}(1, z) = -\log(1 - z)$, but `singularities` never asks for that rewrite.

```
e = expression.rewrite([sec, csc, cot, tan], cos)
e = e.rewrite([sech, csch, coth, tanh], cosh)
for i in e.atoms(Pow):
    ...
for i in expression.atoms(log, asech, acsch):
    sings += solveset(i.args[0], symbol, domain)
for i in expression.atoms(atanh, acoth):
    ...
```

For $\text{polylog}(1, x)$, there are no relevant `Pow` atoms and no visible `log` atoms. The hard-coded scans therefore add no candidate singularities and return the initial `EmptySet`. If the expression is first expanded to $-\log(1 - x)$, the existing logarithm branch solves $1 - x = 0$ and finds $x = 1$, which indicates that the miss is in dispatch or rewriting rather than in solving the logarithm argument. A plausible fix direction supported by the diagnosis is to add a targeted $\text{polylog}(1, z)$ case, or to expand supported special functions before scanning for known singular forms.

1.6 Additional Failing Instances

The same failure occurs for affine arguments $a(x)$ where $a(x) = 1$ has a solution. In each case, $\text{polylog}(1, a(x)) = -\log(1 - a(x))$, so any root of $1 - a(x)$ is a logarithmic singularity. The parametrized evidence covers the following cases.

Input / Expression	SymPy behavior	Expected behavior
$\text{polylog}(1, x)$	Returns <code>EmptySet</code> ; does not contain 1	Singularity set contains 1
$\text{polylog}(1, x + 1)$	Does not contain 0	Singularity set contains 0
$\text{polylog}(1, x - 2)$	Does not contain 3	Singularity set contains 3
$\text{polylog}(1, 2x)$	Does not contain $1/2$	Singularity set contains $1/2$
$\text{polylog}(1, -x)$	Does not contain -1	Singularity set contains -1
$\text{polylog}(1, x/3)$	Does not contain 3	Singularity set contains 3

The expected behavior is uniform across the family because the only change is the inner argument of the same logarithmic identity. When $a(x) = 1$, the equivalent expression $-\log(1 - a(x))$ has logarithm argument zero.

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. The closest public material identified was SymPy documentation noting the general limitations of `singularities`, older public source documentation describing limited supported forms, and external Sage documentation for the $\text{polylog}(1, x)$ logarithmic rewrite. No public SymPy issue, pull request, Stack Overflow report, release note, or mailing-list discussion was identified for the exact call `singularities(polylog(1, x), x, S.Complexes)` returning `EmptySet`. The deduplication verdict is therefore a new failure mode with no public precedent, and the concrete reproducer remains individually actionable as a targeted issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest
from sympy import S, polylog, symbols
from sympy.calculus.singularities import singularities

x = symbols("x")

@pytest.mark.parametrize(
    ("arg", "point"),
    [
        (x, S.One),
        (x + 1, S.Zero),
        (x - 2, S(3)),
        (2*x, S.Half),
        (-x, -S.One),
        (x/3, S(3)),
```

```
    ],  
    )  
def test_singularities_polylog_one_includes_log_singularity(arg, point):  
    found = singularities(polylog(1, arg), x, S.Complexes)  
    assert found.contains(point) == S.true
```